

Apollo Architecture Document

1. Overview

Apollo is a next-generation microservices platform designed to handle real-time data processing at scale. It leverages event-driven architecture, container orchestration, and distributed tracing to provide a robust foundation for modern applications.

1.1 Key Design Goals

- Scalability: Horizontal scaling to handle 100K+ requests per second
- Resilience: Self-healing with circuit breakers and automatic failover
- Observability: Distributed tracing with OpenTelemetry integration
- Security: Zero-trust network with mTLS between all services

1.2 Technology Stack

2. Authentication System

2.1 JWT-Based Authentication

Apollo uses JWT (JSON Web Tokens) for stateless authentication. The authentication flow works as follows:

1. Login: User credentials are sent to the Auth Service
2. Validation: Auth Service validates against the user database
3. Token Generation: A signed JWT is generated with a 15-minute expiry
4. Refresh Token: A long-lived refresh token (7 days) is also issued
5. Token Verification: All subsequent requests verify the JWT via the Auth Service's public key

2.2 API Key Authentication

For machine-to-machine communication, Apollo supports API key authentication:

- API keys are hashed using bcrypt before storage
- Keys are prefixed with ap_ for identification
- Each key has scoped permissions (read, write, admin)
- Keys can be revoked individually without affecting other keys

2.3 OAuth 2.0 Integration

Apollo supports OAuth 2.0 with the following grant types:

- Authorization Code: For web applications
- Client Credentials: For server-to-server communication
- Refresh Token: For maintaining long-lived sessions

3. Data Pipeline

3.1 Event Processing

The event processing pipeline handles millions of events per minute:

Producer ? Kafka Topic ? Stream Processor ? Sink Connector ? Database

Events are serialized using Apache Avro for schema evolution support. Each event carries:

- A unique event ID (UUID v7)
- Timestamp in RFC 3339 format
- Event type and version
- Payload in Avro binary format

3.2 Stream Processing

Apache Flink is used for stream processing with the following operators:

- Filter: Remove invalid or spam events
- Map: Transform event formats
- Window: Aggregate events in tumbling windows
- Join: Correlate events from multiple streams

3.3 Data Retention

Data retention policies are configurable per topic:

4. Deployment Strategy

4.1 Kubernetes Deployment

Apollo runs on Kubernetes with the following deployment strategy:

- Canary Deployments: 10% traffic shifted to new version, then 50%, then 100%
- Rolling Updates: Max surge of 25%, max unavailable of 25%

- Blue-Green: Full environment swap for critical services

4.2 CI/CD Pipeline

The CI/CD pipeline is implemented using GitHub Actions:

1. Build: TypeScript compilation + Docker image build
2. Test: Unit tests, integration tests, e2e tests
3. Security Scan: Snyk vulnerability scanning + SAST
4. Deploy: Automated deployment to staging, manual approval for production

4.3 Infrastructure as Code

All infrastructure is defined using Terraform:

```
module "kubernetes_cluster" {  
  source = "./modules/gke"  
  region = "us-west1"  
  node_count = 3  
  machine_type = "e2-standard-4"  
}
```

5. Monitoring and Observability

5.1 Distributed Tracing

Apollo uses OpenTelemetry for distributed tracing:

- Trace Context: Propagated via W3C Trace Context headers
- Sampling: Head-based sampling at 10% for production
- Export: OTLP exporter sending to Jaeger

5.2 Metrics Collection

Key metrics collected from every service:

- Request latency (p50, p95, p99)
- Error rate (5xx responses / total requests)
- Request rate (requests per second)
- Resource usage (CPU, memory, GC pressure)

5.3 Alerting Rules

Critical alerts are configured in Prometheus:

groups:

- name: apollo-critical

rules:

- alert: HighErrorRate

expr: error_rate > 0.05

for: 5m

severity: critical

- alert: ServiceDown

expr: up == 0

for: 1m

severity: critical

6. API Reference

6.1 REST Endpoints

6.2 WebSocket Events

Apollo supports real-time communication via WebSockets:

- connection.established: Fired when a client connects
- data.updated: Fired when subscribed data changes
- notification.received: Push notifications to clients
- error.occurred: Error events for client-side handling

6.3 Rate Limiting

API rate limits are enforced per tenant:

- Free tier: 100 requests/minute
- Pro tier: 1,000 requests/minute
- Enterprise: 10,000 requests/minute (configurable)

Rate limit headers are returned in every response:

X-RateLimit-Limit: 1000

X-RateLimit-Remaining: 987

X-RateLimit-Reset: 1625097600

7. Performance Benchmarks

7.1 Latency Tests

Tests performed on a 3-node cluster (e2-standard-4):

7.2 Throughput

Under sustained load testing:

- API Gateway: 15,000 requests/second per instance
- Event Pipeline: 50,000 events/second per partition
- Database Reads: 8,000 queries/second (with connection pooling)
- Database Writes: 2,500 writes/second (with batch inserts)

8. Security Considerations

8.1 Encryption

- In Transit: TLS 1.3 for all HTTP traffic, mTLS for service-to-service
- At Rest: AES-256 encryption for all data stores
- Key Management: AWS KMS for key rotation and management

8.2 Access Control

Role-based access control (RBAC) with three built-in roles:

- Admin: Full access to all resources
- Developer: Read/write access to development resources
- Viewer: Read-only access to monitoring and logs

8.3 Audit Logging

All security-relevant events are logged:

- Authentication attempts (success and failure)
- API key creation and revocation
- Permission changes
- Data export operations

9. Getting Started

9.1 Local Development

To set up a local development environment:

Clone the repository

```
git clone https://github.com/apollo/platform.git
```

```
cd platform
```

Install dependencies

npm install

Start development services

`docker compose up -d`

Run database migrations

```
npm run migrate
```

Start the development server

`npm run dev`

9.2 Configuration

Configuration is managed via environment variables:

Database

DATABASE_URL=postgresql://localhost:5432/apollo

REDIS_URL=redis://localhost:6379

Authentication

JWT_SECRET=your-secret-key

JWT_EXPIRY=15m

REFRESH_TOKEN_EXPIRY=7d

Kafka

KAFKA_BROKERS=localhost:9092

KAFKA_GROUP_ID=apollo-processor

10. Troubleshooting Guide

10.1 Common Issues

Issue: Connection refused to database

- Check if PostgreSQL is running: `docker ps | grep postgres`
- Verify connection string in environment variables
- Ensure network policies allow traffic on port 5432

Issue: JWT token expired

- Tokens expire after 15 minutes
- Use the refresh token endpoint to get a new token
- Check system clock synchronization (NTP)

Issue: High memory usage

- Review the garbage collection logs
- Increase heap size: `--max-old-space-size=4096`
- Check for memory leaks in event handlers

10.2 Debugging

Enable debug logging by setting the LOG_LEVEL environment variable:

LOG_LEVEL=debug npm run dev

Debug logs include:

- Incoming request details
- Database query execution plans
- Event processing latency breakdowns
- External API call timing